

Praktika 1: Aktoreak eta filmak kudeatzeko aplikazioa

Maialen Ruiz Gallo
Ibon Madariaga Beldarrain
Endika Perez Larrea
Marco Alonso Fernandez

October 8, 2026

Contents

1	Sarrera: Arazoa	3
2	Klaseen diseinua	4
3	Datu-egitura nagusien deskribapena	5
3.1	<i>ArrayList</i>	5
3.2	<i>Hash</i> taula	5
3.3	Konparaketa	5
4	Metodo nagusien diseinua eta implementazioa	6
4.1	main metodoa	6
4.2	readFiles metodoa	7
4.3	readFile metodoa	8
4.4	Quick Sort algoritmoa	9
4.4.1	quickSortAlgorithm metodoa	9
4.4.2	quickSort metodoa	10
4.4.3	divisor metodoa	11
4.4.4	swap metodoa	11
5	Kodigoa	13
5.1	Main klasea	13
5.2	ReadFile klasea	14
5.3	WriteFile klasea	16
5.4	QuickSort klasea	17
5.5	FilmZerrenda klasea	18
5.6	FilmTaula klasea	19
5.7	Film klasea	20
5.8	AktoreZerrenda klasea	21
5.9	AktoreTaula klasea	22
5.10	Aktore klasea	23
5.11	JUNIT klaseak	24
5.11.1	AktoreTest klasea	24
5.11.2	AktoreZerrendaTest klasea	25
5.11.3	AktoreTaulaTest klasea	26
5.11.4	FilmTest klasea	27
5.11.5	FilmZerrendaTest klasea	28
5.11.6	FilmTaulaTest klasea	28

6	Konklusioak	30
6.1	Saiakerak eta emaitzak	30
6.2	ArrayList eta Hash taularen konparaketa grafikoa	31
6.3	Quick Sort algoritmoaren exekuzio denbora milisegunduetan (ms)	31

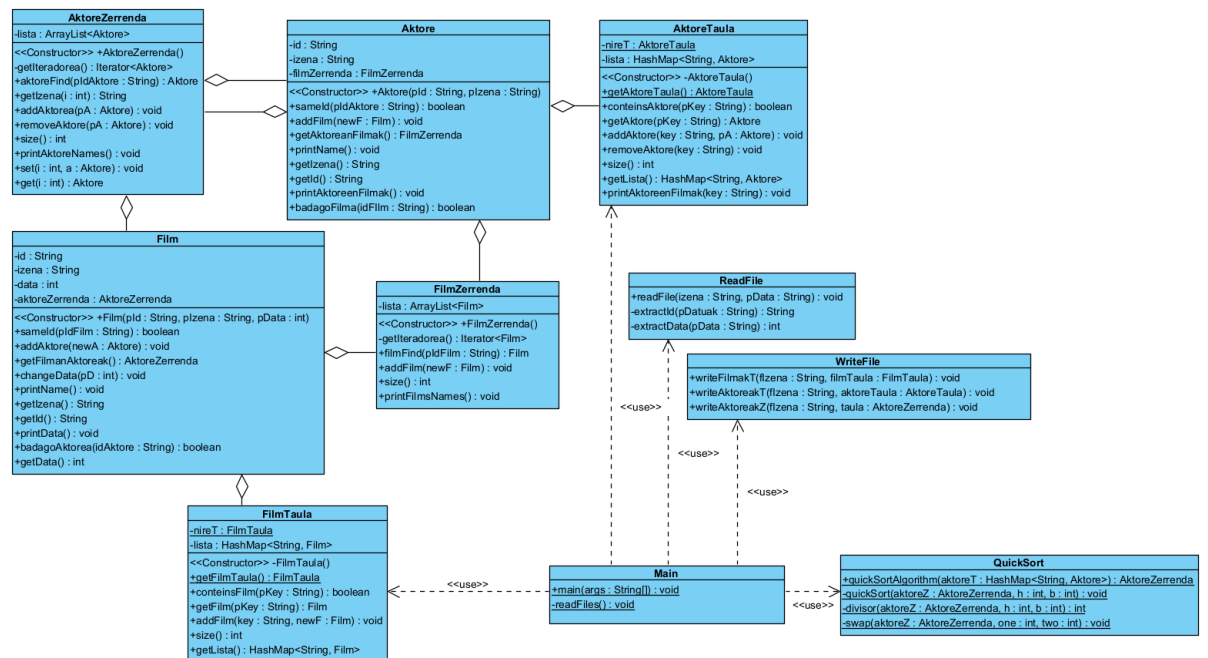
Chapter 1

Sarrera: Arazoa

Aplikazio bat garatu nahi da, aktore-kopuru handi bat eta parte hartzen duten filmak kudeatuko dituen. Datuak Wikidatatik jaso dira. Aktore batek film askotan parte har dezake. Era berean, film batean hainbat aktorek lan egin dezakete.

Chapter 2

Klaseen diseinua



Chapter 3

Datu-egitura nagusien deskribapena

Aurretik aipatutako arazoa konpontzeko, bi datu-egitura erabili dira: *ArrayList*-ak eta *Hash* taulak.

Hasieran, *ArrayList*-ak erabili ziren soilik, baina algoritmoa ez zen hain eraginkorra. Hori dela eta, *Hash* taula bat erabiltzea erabaki zen.

3.1 *ArrayList*

ArrayList Java-ren datu-egitura bat da, *array*-etatik eratorritakoa. Elementuak elkarren alboan gordeta gelditzen dira memorian eta betetzean, array berri bat sortzen da, tamainaz bikoitza izan ohi dena, eta bertara kopiatzen dira elementu guztiak (orden konstanteko eragiketa da gehienetan, baina betetzean, n -ko kostua du).

3.2 *Hash* taula

Hash taulak ere bai arrayetatik datoz. Hash tauletan, elementu bat sartzean, funtzio bat aplikatzen da arrayaren indizea kalkulatzeko eta bertan gordeko da. *ArrayList* ez bezala, Hash taulak ez du ordenik bermatzen, hots, ordena alda daiteke.

3.3 Konparaketa

Orokorrean, Hash taulak *ArrayList* baino eraginkorragoa da. Esaterako, balio bat bilatzeko, *ArrayList*ean $O(n)$ da eta hash taulan $O(1)$. Hala ere, hash taulak ez du NULL baliorik onartzen. *ArrayList*ek, aldiz, bai.

Chapter 4

Metodo nagusien diseinua eta implementazioa

4.1 main metodoa

Espezifikazioa

- **Aurrebaldintza:** Command-Line Argument-ko argumentuak hartzen ditu parametro gisa, baina ez dira erabiltzen. Eclipsek defektuz sortutako metodo izenburua da.
- **Postbaldintza:** Ez du ezer itzultzen

Deskribapena

Metodo hau nagusia da. Bertan, bi hash taula lortzen dira. Horretaz aparte, Quick Sort algoritmoa bertatik deitzen da eta exekuzio-denborak kontrolatu egiten dira.

Kostua

$$\begin{aligned} T(n) &= t_{has} + t_{readFiles} + t_{has1} + t_{writeF} + t_{writeAkt} + t_{has2} + t_{QuickSort} + t_{writeSorted} + t_{has3} = \\ &= K_1 + t_{readFiles} + t_{writeF} + t_{writeAkt} + t_{QuickSort} + t_{writeSorted} = \\ &= K_1 + n^3 + 3n + n \log_2(n) \in O(n^3) \end{aligned}$$

Beraz, main metodoaren ordena **kubikoa** da.

Implementazioa

```
public static void main(String[] args) {
    long start = System.currentTimeMillis();
    //ALGORITMO NAGUSIA
    AktoreTaula aktoreZ = AktoreTaula.getAktoreTaula();
    FilmTaula filmZ = FilmTaula.getFilmTaula();
    readFiles(filmZ.getLista(), aktoreZ.getLista());
}
```

```

//EXEKUZIO DENBORA
long end = System.currentTimeMillis();
double exTime = (double) ((end - start)/1000);
//PRINT-AK
System.out.println("execution_time: " + exTime + "
seconds");
System.out.println("AktoreZ " + aktoreZ.size());
System.out.println("FilmZ " + filmZ.size());
//IDAZKETA
WriteFile writeFile = new WriteFile();
//writeFile.writeFilmakT("filmak.txt", filmZ.getList())
;
//writeFile.writeAktoreakT("aktoreak.txt", aktoreZ.
getList());
//SORT
AktoreZerrenda aktoreSorted = QuickSort.
quickSortAlgorithm(aktoreZ.getList());
writeFile.writeAktoreakZ("aktoreakSorted.txt",
aktoreSorted);
}

```

4.2 readFiles metodoa

Espezifikazioa

- **Aurrebaldintza:** Parametro gisa, aktore guztiak gordeta dauden (edo gordeko diren lekua) taula eta film guztiak gordeta daude (edo gordeko diren lekua) taula jasotzen ditu.
- **Postbaldintza:** Metodoak ez du ezer itzultzen.

Deskribapena

Metodo honek .txt artxibo guztiak array baten barnean sartzen ditu eta artxiboz artxibo irakurtzen ditu.

Kostua

$$\begin{aligned}
T(n) &= t_{has} + t_{listFiles} + t_{for} = K_1 + n + n(t_{bald} + t_{readFile}) = \\
&= K_1 + n + n(K_2 + n^2) = K_1 + n + nK_2 + n^3 = \\
&= K_1 + K_3n + n^3 \in O(n^3)
\end{aligned}$$

Beraz, readFiles metodoaren ordena **kubikoa** da.

Implementazioa

```

private static void readFiles(HashMap<String, Film> filmZ,
HashMap<String, Aktore> aktoreZ) {

```

```

File directory = new File("C:/Users/maier/OneDrive-UPV
    EHU/2.maila/1.kuatri/datu-egiturak_eta_algoritmoak/
    Praktikak/Aktoreak_Filmak/src/packLaboTaula/movies-
    dir/");
File[] files = directory.listFiles(); // Array batean
    gorde .txt guztiak
ReadFile reader = new ReadFile();
for(File file : files) {
    if (file.isFile() && file.getName().endsWith(".txt"))
        reader.readFile(file.getAbsolutePath(), file.getName()
            , filmZ, aktoreZ);
}
}

```

4.3 readFile metodoa

Espezifikazioa

- **Aurrebaldintza:** Artxiboaren "Path"-a, izena eta aktore eta filmen taulak hartzen ditu parametro gisa.
- **Postbaldintza:** Metodoak ez du ezer itzultzen.

Deskribapena

Metodo hau Main klaseko readFiles metodotik deitzen da iterazio bakoitzean. Bertan, .txt bakoitzaren irakurketa bera burutzen da. Artxiboa lerroz lerro irakurri egiten da, bakoitzetik beharrezko informazioa lortuz eta dagokion lekuak sailkatuz.

Kostua

$$\begin{aligned}
 T(n) &= t_{has1} + t_{while} + t_{has2} = K_1 + t_{while} = \\
 &= K_1 + n(t_{bald} + t_{badagoF} + t_{badagoAkt}) = K_1 + n(K_2 + n + n) = \\
 &= K_1 + n(K_2 + 2n) = K_1 + K_2n + 2n^2 \in O(n^2)
 \end{aligned}$$

Beraz, readFile metodoaren ordena **koadratikoa** da.

Implementazioa

```

public void readFile(String izena, String pData, HashMap<
    String, Film> filmZ, HashMap<String, Aktore> aktoreZ) {
    try {
        Scanner sarrera = new Scanner(new FileReader(izena));
        String lerroa;
        while (sarrera.hasNext()) {
            lerroa = sarrera.nextLine();
            String[] datuak = lerroa.split("\\s+###\\s+");
            int data = extractData(pData);

```

```

String aktoreKey = datuak[0].concat(datuak[1]);
String filmKey = datuak[2].concat(datuak[3]);
if (datuak.length == 4) {
    datuak[0] = extractId(datuak[0]);
    datuak[2] = extractId(datuak[2]);
    if (!(datuak[0].equals(datuak[1]) || datuak[2].
        equals(datuak[3]))) {
        if (!aktoreZ.containsKey(aktoreKey)) {
            Aktore newA = new Aktore(datuak[0], datuak[1]);
            aktoreZ.put(aktoreKey, newA);
        }
        if (!filmZ.containsKey(filmKey)) {
            Film newF = new Film(datuak[2], datuak[3], data)
            ;
            filmZ.put(filmKey, newF);
        }
        aktoreZ.get(aktoreKey).addFilm(filmZ.get(filmKey))
        ;
        filmZ.get(filmKey).addAktore(aktoreZ.get(aktoreKey)
        ));
    }
}
}
sarrera.close();
}
catch (IOException e) {
    e.printStackTrace();
}
}

```

4.4 Quick Sort algoritmoa

4.4.1 quickSortAlgorithm metodoa

Espezifikazioa

- **Aurrebaldintza:** Metodoak aktoreen hash taula hartzen du parametro gisa.
- **Postbaldintza:** Metodoak ArrayList bat itzultzen du. Bertan, taulako aktore guztiak agertzen dira, ordenatuta.

Deskribapena

Metodo honek algoritmoaren hasieraketak egiten ditu, hala nola, zerrenda berri bat sortu, algoritmoa burutzen duen metodoari deitu eta zerrenda ordenatua itzuli.

Kostua

$$T(n) = t_{has} + t_{for} + t_{quickSort} + t_{ret} = K_1 + t_{for} + t_{quickSort} =$$

$$= K_1 + nK_2 + t_{div} * \log_2(n) = K_1 + nK_2 + n \log_2(n) \in O(n \log_2(n))$$

Kasu orokorra hartzen bada kontuan (txarrena baino), Quick Sort algoritmoaren ordena **kuasilineala** da.

Implementazioa

```
public static AktoreZerrenda quickSortAlgorithm(HashMap<
    String, Aktore> AktoreT) {
    AktoreZerrenda sortedList = new AktoreZerrenda();
    for (String key: AktoreT.keySet()) {
        sortedList.addAktorea(AktoreT.get(key));
    }
    quickSort(sortedList, 0, sortedList.size()-1);
    return sortedList;
}
```

4.4.2 quickSort metodoa

Espezifikazioa

- **Aurrebaldintza:** Metodoak aktore-taula eta bi zenbaki oso jasotzen ditu (hasierako eta bukaerako posizioak).
- **Postbaldintza:** Metodoak ez du ezer itzultzen.

Deskribapena

Metodo honek quick sort algoritmoa bera burutzen du, dei errekurtsiboekin.

Kostua

$$T(n) = t_{div} + \log_2(n) = n \log_2(n) \in O(n \log_2(n))$$

Beraz, Quick Sort algoritmoaren ordena **kuasilineala** da.

Implementazioa

```
private static void quickSort(AktoreZerrenda aktoreZ, int
    h, int b) {
    if (b-h > 0) {
        int index = divisor(aktoreZ, h, b);
        quickSort(aktoreZ, h, index-1);
        quickSort(aktoreZ, index+1, b);
    }
}
```

4.4.3 divisor metodoa

Espezifikazioa

- **Aurrebaldintza:** Metodoak aktore-aula eta bi zenbaki oso jasotzen ditu (hasierako eta bukaerako posizioak).
- **Postbaldintza:** Metodoak ez du ezer itzultzen.

Deskribapena

Metodo honek Quick Sort algoritmoak behin eta berriz egiten duen zatiketa burutzen du.

Kostua

$$T(n) = t_{has} + t_{while} + t_{ret} = K_1 + t_{while} = K_1 + nK_2 \in O(n)$$

Beraz, divisor metodoaren ordena **lineala** da.

Implementazioa

```
private static int divisor(AktoreZerrenda aktoreZ, int h,
    int b) {
    Aktore tempAktore = aktoreZ.get(h);
    String tempIzena = aktoreZ.getIzena(h);
    int left = h;
    int right = b;
    while (left < right) {
        while (aktoreZ.getIzena(left).compareTo(tempIzena) <=
            0 && left < right){
            left++;
        }
        while (aktoreZ.getIzena(right).compareTo(tempIzena) >
            0 && left < right){
            right--;
        }
        if (left < right)
            swap(aktoreZ, left, right);
    }
    if (aktoreZ.getIzena(right).compareTo(tempIzena) > 0)
        right--;
    aktoreZ.set(h, aktoreZ.get(right));
    aktoreZ.set(right, tempAktore);
    return right;
}
```

4.4.4 swap metodoa

Espezifikazioa

- **Aurrebaldintza:** Metodoak aktore-aula eta bi zenbaki oso jasotzen ditu (trukatu nahi diren bi elementuen posizioak).
- **Postbaldintza:** Metodoak ez du ezer itzultzen.

Deskribapena

Metodo honek bi balioen aldaketa burutzen du, aldagai temporal baten laguntzarekin.

Kostua

Kostua konstantea da $O(1)$ edozein posizioko edozein elementurekin denbora berdina behar izango baitu.

Implementazioa

```
private static void swap(AktoreZerrenda aktoreZ, int one,
    int two) {
    Aktore tempAktore = aktoreZ.get(one);
    aktoreZ.set(one, aktoreZ.get(two));
    aktoreZ.set(two, tempAktore);
}
```

Chapter 5

Kodigoa

5.1 Main klasea

```
package packLaboTaula;

import java.io.File;

public class Main {

    public static void main(String[] args) {
        long start = System.currentTimeMillis();
        //ALGORITMO NAGUSIA
        AktoreTaula aktoreZ = AktoreTaula.getAktoreTaula();
        FilmTaula filmZ = FilmTaula.getFilmTaula();
        readFiles();
        //EXEKUZIO DENBORA: ALGORITMO NAGUSIA
        long end = System.currentTimeMillis();
        double exTime = (double) ((end - start)/1000);

        //PRINT-AK
        System.out.println("===PRINT===\n");
        System.out.println("execution_time_ algorithm_
            nagusia:" + exTime + " seconds");
        System.out.println("AktoreZ_ " + aktoreZ.size());
        System.out.println("FilmZ_ " + filmZ.size());
        System.out.println("");
        //IDAZKETA
        WriteFile writeFile = new WriteFile();
        writeFile.writeFilmakT("filmak.txt", FilmTaula.
            getFilmTaula());
        writeFile.writeAktoreakT("aktoreak.txt", AktoreTaula.
            getAktoreTaula());

        //SORT
        System.out.println("===SORT===\n");
        start = System.currentTimeMillis();
        AktoreZerrenda aktoreSorted = QuickSort.
            quickSortAlgorithm(aktoreZ.getList());
```

```

        writeFile.writeAktoreakZ("aktoreakSorted.txt",
            aktoreSorted);
        end = System.currentTimeMillis();
        exTime = (double) ((end - start)/100);
        System.out.println("execution_time-quick_sort: " +
            exTime + " milliseconds");
        System.out.println("");

        //PROBAK
        System.out.println("===PROBAK===\n");
        aktoreZ.printAktoreenFilmak("HarrisonFordQ81328");
        System.out.println("");
        aktoreZ.printAktoreenFilmak("HarryStylesQ3626966");
        System.out.println("");
        aktoreZ.printAktoreenFilmak("ZendayaQ189489");
    }

    private static void readFiles() {
        File directory = new File("C:/Users/maier/Desktop/
            DEA-proiektua/Praktikak/Aktoreak_Filmak/src/
            packLaboTaula/movies-dir/");
        File[] files = directory.listFiles();
        ReadFile reader = new ReadFile();
        for(File file : files) {
            if (file.isFile() && file.getName().endsWith(".txt
                "))
                reader.readFile(file.getAbsolutePath(), file.
                    getName());
        }
    }
}

```

5.2 ReadFile klasea

```

package packLaboTaula;

import java.io.FileReader;
import java.io.IOException;
import java.util.Scanner;

public class ReadFile {
    public void readFile(String izena, String pData) {
        try {
            Scanner sarrera = new Scanner(new FileReader(izena))
                ;
            int data = extractData(pData);
            while (sarrera.hasNext()) {
                String[] datuak = sarrera.nextLine().split("\\s
                    +###\\s+");
                if (datuak.length != 4) continue;
                String aktoreId = extractId(datuak[0]);
                String aktoreIzena = datuak[1];
            }
        } catch (IOException e) {
            e.printStackTrace();
        }
    }
}

```

```

        String filmId = extractId(datuak[2]);
        String filmIzena = datuak[3];

        if (aktoreId.equals(aktoreIzena) || filmId.equals(
            filmIzena)) continue;

        String aktoreKey = aktoreIzena.replace(" ", "").
            concat(aktoreId);
        String filmKey = filmIzena.replace(" ", "").concat
            (filmId);

        Aktore aktore = AktoreTaula.getAktoreTaula().
            getAktore(aktoreKey);
        if (aktore == null) {
            aktore = new Aktore(aktoreId, aktoreIzena);
            AktoreTaula.getAktoreTaula().addAktore(aktoreKey
                , aktore);
        }

        Film film = FilmTaula.getFilmTaula().getFilm(
            filmKey);
        if (film == null) {
            film = new Film(filmId, filmIzena, data);
            FilmTaula.getFilmTaula().addFilm(filmKey, film);
        }

        if(!aktore.badagoFilma(filmId)) aktore.addFilm(
            film);
        if(!film.badagoAktorea(aktoreId)) film.addAktore(
            aktore);
    }
    sarrera.close();
} // try
catch (IOException e) {
    e.printStackTrace();
}
}

private String extractId(String pDatuak) {
    String[] datuak = pDatuak.split("/");
    return (datuak[4]);
}

private int extractData(String pData) {
    String[] dataSplit = pData.split("_");
    String[] dataSplit2 = dataSplit[3].split("\\.");
    int data = Integer.parseInt(dataSplit2[0]);
    return data;
}
}

```

5.3 WriteFile klasea

```
package packLaboTaula;

import java.io.IOException;
import java.io.PrintWriter;

public class WriteFile {
    public void writeFilmaKT(String fIzena, FilmTaula
        filmTaula) {
        try {
            PrintWriter writer = new PrintWriter(fIzena, "UTF-8"
                );
            for (Film film : filmTaula.getLista().values())
                writer.printf("%-60s□□%s%n", film.getIzena(), film.
                    getId());
            writer.close();
        }
        catch (IOException e) {
            e.printStackTrace();
        }
    }

    public void writeAktoreakT(String fIzena, AktoreTaula
        aktoreTaula) {
        try {
            PrintWriter writer = new PrintWriter(fIzena, "UTF-8"
                );
            for (Aktore aktore : aktoreTaula.getLista().values()
                )
                writer.printf("%-60s□□%s%n", aktore.getIzena(),
                    aktore.getId());
            writer.close();
        }
        catch (IOException e) {
            e.printStackTrace();
        }
    }

    public void writeAktoreakZ(String fIzena, AktoreZerrenda
        taula) {
        try {
            PrintWriter writer = new PrintWriter(fIzena, "UTF-8"
                );
            for (int i = 0; i < taula.size() ; i++)
                writer.printf("%-60s□□%s%n", taula.get(i).getIzena()
                    , taula.get(i).getId());
            writer.close();
        }
        catch (IOException e) {
            e.printStackTrace();
        }
    }
}
```

5.4 QuickSort klasea

```
package packLaboTaula;

import java.util.HashMap;

public class QuickSort {

    public static AktoreZerrenda quickSortAlgorithm(HashMap<
        String, Aktore> AktoreT) {
        AktoreZerrenda sortedList = new AktoreZerrenda();
        for (String key: AktoreT.keySet()) {
            sortedList.addAktorea(AktoreT.get(key));
        }
        //System.out.print(sortedList.getIzena(0) +"      "+
            sortedList.getIzena(1));
        //System.out.print(sortedList.getIzena(0).compareTo(
            sortedList.getIzena(1)));
        quickSort(sortedList, 0, sortedList.size()-1);
        return sortedList;
    }

    private static void quickSort(AktoreZerrenda aktoreZ,
        int h, int b) {
        if (b-h > 0) {
            int index = divisor(aktoreZ, h, b);
            quickSort(aktoreZ, h, index-1);
            quickSort(aktoreZ, index+1, b);
        }
    }

    private static int divisor(AktoreZerrenda aktoreZ, int h
        , int b) {
        Aktore tempAktore = aktoreZ.get(h);
        String tempIzena = aktoreZ.getIzena(h);
        int left = h;
        int right = b;
        while (left < right) {
            while (aktoreZ.getIzena(left).compareTo(tempIzena)
                <= 0 && left < right)
                left++;
            while (aktoreZ.getIzena(right).compareTo(tempIzena)
                > 0 && left < right)
                right--;
            if (left < right)
                swap(aktoreZ, left, right);
        }
        if (aktoreZ.getIzena(right).compareTo(tempIzena) > 0)
            right--;
        aktoreZ.set(h, aktoreZ.get(right));
        aktoreZ.set(right, tempAktore);
        return right;
    }
}
```

```

        private static void swap(AktoreZerrenda aktoreZ, int one
            , int two) {
            Aktore tempAktore = aktoreZ.get(one);
            aktoreZ.set(one, aktoreZ.get(two));
            aktoreZ.set(two, tempAktore);
        }
    }
}

```

5.5 FilmZerrenda klasea

```

package packLaboTaula;

import java.util.ArrayList;
import java.util.Iterator;

public class FilmZerrenda {
    private ArrayList<Film> lista;

    public FilmZerrenda() {
        this.lista = new ArrayList<Film>();
    }

    private Iterator<Film> getIteradorea(){
        return this.lista.iterator();
    }

    public Film filmFind(String pIdFilm)
    {
        Film filmFound = null;
        Film f = null;
        Iterator<Film> itr = this.getIteradorea();
        while(itr.hasNext() && filmFound == null) {
            f = itr.next();
            if (f.sameId(pIdFilm))
                filmFound = f;
        }
        return filmFound;
    }

    public void addFilm(Film newF) {
        this.lista.add(newF);
    }

    public int size() {
        return this.lista.size();
    }

    public void printFilmsNames() {
        Film f = null;
        Iterator<Film> itr = this.getIteradorea();
        while(itr.hasNext()) {
            f = itr.next();

```

```

        System.out.print("    ");
        f.printName();
        System.out.print("    ");
        f.printData();
        System.out.println();
    }
}
}

```

5.6 FilmTaula klasea

```

package packLaboTaula;

import java.util.HashMap;

public class FilmTaula {
    private static FilmTaula nireT = null;
    private HashMap<String, Film> lista;

    private FilmTaula() {
        this.lista = new HashMap<String, Film>();
    }

    public static FilmTaula getFilmTaula() {
        if (nireT == null)
            nireT = new FilmTaula();
        return nireT;
    }

    public boolean conteinsFilm(String pKey)
    {
        return lista.containsKey(pKey);
    }

    public Film getFilm(String pKey) {
        return this.lista.get(pKey);
    }

    public void addFilm(String key, Film newF) {
        this.lista.put(key, newF);
    }

    public int size() {
        return this.lista.size();
    }

    public HashMap<String, Film> getLista(){
        return this.lista;
    }
}

```

5.7 Film klasea

```
package packLaboTaula;

public class Film {
    private String id, izena;
    private int data;
    private AktoreZerrenda aktoreZerrenda;

    public Film(String pId, String pIzena, int pData) {
        this.id = pId;
        this.izena = pIzena;
        this.data = pData;
        this.aktoreZerrenda = new AktoreZerrenda();
    }

    public boolean sameId(String pIdFilm) {
        if (this.id.equals(pIdFilm))
            return true;
        return false;
    }

    public void addAktore(Aktore newA) {
        this.aktoreZerrenda.addAktorea(newA);
    }

    public AktoreZerrenda getFilmanAktoreak() {
        return this.aktoreZerrenda;
    }

    public void changeData(int pD) {
        this.data = pD;
    }

    public void printName() {
        System.out.print(this.izena);
    }

    public String getIzena() {
        return this.izena ;
    }

    public String getId() {
        return this.id ;
    }

    public void printData() {
        System.out.print(this.data);
    }

    public boolean badagoAktorea(String idAktore) {
        if (this.aktoreZerrenda.aktoreFind(idAktore) != null)
            return true;
        return false;
    }
}
```

```

    }

    public int getData() {
        return this.data;
    }

}

```

5.8 AktoreZerrenda klasea

```

package packLaboTaula;

import java.util.ArrayList;
import java.util.Iterator;

public class AktoreZerrenda {
    private ArrayList<Aktore> lista;

    public AktoreZerrenda() {
        this.lista = new ArrayList<Aktore>();
    }

    private Iterator<Aktore> getIteradorea(){
        return this.lista.iterator();
    }

    public Aktore aktoreFind(String pIdAktore)
    {
        Aktore aktoreFound = null;
        Aktore f = null;
        Iterator<Aktore> itr = this.getIteradorea();
        while(itr.hasNext() && aktoreFound == null) {
            f = itr.next();
            if (f.sameId(pIdAktore))
                aktoreFound = f;
        }
        return aktoreFound;
    }

    public String getIzena(int i) {
        return this.lista.get(i).getIzena();
    }

    public void addAktorea(Aktore pA) {
        this.lista.add(pA);
    }

    public void removeAktore(Aktore pA) {
        this.lista.remove(pA);
    }

    public int size() {

```

```

        return this.lista.size();
    }

    public void printAktoreNames() {
        Aktore f = null;
        Iterator<Aktore> itr = this.getIteradorea();
        while(itr.hasNext()) {
            f = itr.next();
            f.printName();
        }
    }

    public void set(int i, Aktore a) {
        this.lista.set(i, a);
    }

    public Aktore get(int i) {
        return this.lista.get(i);
    }
}

```

5.9 AktoreTaula klasea

```

package packLaboTaula;

import java.util.HashMap;

public class AktoreTaula {
    private static AktoreTaula nireT = null;
    private HashMap<String, Aktore> lista;

    private AktoreTaula() {
        this.lista = new HashMap<String, Aktore>();
    }

    public static AktoreTaula getAktoreTaula() {
        if (nireT == null)
            nireT = new AktoreTaula();
        return nireT;
    }

    public boolean conteinsAktore(String pKey)
    {
        if (this.lista.containsKey(pKey)) return true;
        else return false;
    }

    public Aktore getAktore(String pKey) {
        return this.lista.get(pKey);
    }
}

```

```

    public void addAktore(String key, Aktore pA) {
        this.lista.put(key, pA);
    }

    public void removeAktore(String key) {
        this.lista.remove(key);
    }

    public int size() {
        return this.lista.size();
    }

    public HashMap<String, Aktore> getList() {
        return lista;
    }

    public void printAktoreenFilma(String key) {
        this.lista.get(key).printAktoreenFilma();
    }
}

```

5.10 Aktore klasea

```

package packLaboTaula;

public class Aktore {
    private String id, izena;
    private FilmZerrenda filmZerrenda;

    public Aktore(String pId, String pIzena) {
        this.id = pId;
        this.izena = pIzena;
        this.filmZerrenda = new FilmZerrenda();
    }

    public boolean sameId(String pIdAktore) {
        if (this.id.equals(pIdAktore))
            return true;
        return false;
    }

    public void addFilm(Film newF) {
        this.filmZerrenda.addFilm(newF);
    }

    public FilmZerrenda getAktoreanFilma() {
        return this.filmZerrenda;
    }

    public void printName() {
        System.out.println(this.izena);
    }
}

```

```

    }

    public String getIzena() {
        return this.izena;
    }

    public String getId() {
        return this.id;
    }

    public void printAktoreenFilma() {
        System.out.println(this.izena + ":");
        this.filmZerrenda.printFilmsNames();
    }

    public boolean badagoFilma(String idFilm) {
        if (this.filmZerrenda.filmFind(idFilm) != null)
            return true;
        return false;
    }
}

```

5.11 JUNIT klaseak

5.11.1 AktoreTest klasea

```

package packLaboTaula;

import static org.junit.jupiter.api.Assertions.*;
import org.junit.jupiter.api.Test;

class AktoreTest {

    Aktore a = new Aktore("QR6767", "Ramon");
    Aktore b = new Aktore("QR6767", "Ignacio");
    Film f = new Film("QT3333", "Frankenstein", 1967);

    //Hacer esto pero en varios @Test, importante
    @Test
    void testGetIzena() {
        assertEquals("Ramon", a.getIzena());
    }

    @Test
    void testGetId() {
        assertEquals("QR6767", a.getId());
    }

    @Test
    void getAktoreanFilmaNotNull() {
        assertNotNull(a.getAktoreanFilma());
    }
}

```

```

@Test
void testSameId() {
    assertTrue(a.sameId(b.getId()));
}

@Test
void testAddFilm() {
    a.addFilm(f);
    assertEquals(a.getAktoreanFilmak().size(),1);
}

@Test
void testBadagoFilma() {
    a.addFilm(f);
    assertTrue(a.badagoFilma(f.getId()));
}
}

```

5.11.2 AktoreZerrendaTest klasea

```

package packLaboTaula;

import static org.junit.jupiter.api.Assertions.*;

import org.junit.jupiter.api.Test;

class AktoreZerrendaTest {

    AktoreZerrenda az = new AktoreZerrenda();
    Aktore a = new Aktore("QR6767","Ramon");
    Aktore a2 = new Aktore("QR6869","Andeka");

    @Test
    void testAddAktore() {
        az.addAktorea(a);
        assertEquals(az.size(),1);
    }

    @Test
    void testRemoveAktore() {
        az.addAktorea(a);
        az.removeAktore(a);
        assertEquals(az.size(),0);
    }

    @Test
    void testAktoreFind() {
        az.addAktorea(a);
        assertEquals(az.aktoreFind(a.getId()),a);
    }

    @Test

```

```

    void testGet() {
        az.addAktorea(a);
        assertEquals(az.get(0),a);
    }

    @Test
    void testSet() {
        az.addAktorea(a);
        az.addAktorea(a2);
        az.set(1, a);
        assertEquals(az.get(1),a);
    }
}

```

5.11.3 AktoreTaulaTest klasea

```

package packLaboTaula;

import static org.junit.jupiter.api.Assertions.*;

import java.util.HashMap;

import org.junit.jupiter.api.Test;

class AktoreTaulaTest {

    Aktore a = new Aktore("QR6767","Ramon");

    @Test
    void testGetAktoreTaula() {
        assertNotNull(AktoreTaula.getAktoreTaula());
    }

    @Test
    void testSize() {
        assertEquals(0, AktoreTaula.getAktoreTaula().size());
    }

    @Test
    void testAddAktore() {
        AktoreTaula.getAktoreTaula().addAktore(a.getId(), a);
        assertEquals(1, AktoreTaula.getAktoreTaula().size());
    }

    @Test
    void testRemoveAktore() {
        AktoreTaula.getAktoreTaula().addAktore(a.getId(), a);
        AktoreTaula.getAktoreTaula().removeAktore(a.getId());
        assertEquals(0, AktoreTaula.getAktoreTaula().size());
    }

    @Test

```

```

void testConteinsAktore() {
    AktoreTaula.getAktoreTaula().addAktore(a.getId(), a);
    assertTrue(AktoreTaula.getAktoreTaula().conteinsAktore
        (a.getId()));
}

@Test
void testGetLista() {
    HashMap<String, Aktore> lista = AktoreTaula.
        getAktoreTaula().getLista();
    assertNotNull(lista);
}
}

```

5.11.4 FilmTest klasea

```

package packLaboTaula;

import static org.junit.jupiter.api.Assertions.*;
import org.junit.jupiter.api.Test;

class FilmTest {

    Film f = new Film("QL6869", "La_Momia", 1998);
    Film f2 = new Film("QL6869", "American_History_X", 1998);
    Aktore a = new Aktore("QR6767", "Ramon");

    //Hacer esto pero en varios @Test, importante para saber
    //donde es el fallo en caso de haber uno

    @Test
    void testGetIzena() {
        assertEquals("La_Momia", f.getIzena());
    }

    @Test
    void testGetId() {
        assertEquals("QL6869", f.getId());
    }

    @Test
    void testGetFilmanAktoreakNotNull() {
        assertNotNull(f.getFilmanAktoreak());
    }

    @Test
    void testSameId() {
        assertTrue(f.sameId(f2.getId()));
    }

    @Test
    void testAddAktore() {

```

```

        f.addAktore(a);
        assertEquals(f.getFilmanAktoreak().size(),1);
    }

    @Test
    void testBadagoAktorea() {
        f.addAktore(a);
        assertTrue(f.badagoAktorea(a.getId()));
    }

    @Test
    void testGetData() {
        assertEquals(f.getData(), 1998);
    }
}

```

5.11.5 FilmZerrendaTest klasea

```

package packLaboTaula;

import static org.junit.jupiter.api.Assertions.*;

import org.junit.jupiter.api.Test;

class FilmZerrendaTest {

    FilmZerrenda fz = new FilmZerrenda();
    Film f = new Film("QL6869","American History X", 1998);

    @Test
    void testSize() {
        assertEquals(fz.size(),0);
    }

    @Test
    void testAddFilm() {
        fz.addFilm(f);
        assertEquals(fz.size(),1);
    }

    @Test
    void testFilmFind() {
        fz.addFilm(f);
        assertEquals(fz.filmFind(f.getId()),f);
    }
}

```

5.11.6 FilmTaulaTest klasea

```

package packLaboTaula;

import static org.junit.jupiter.api.Assertions.*;

```

```

import java.util.HashMap;

import org.junit.jupiter.api.Test;

class FilmTaulaTest {

    Film f = new Film("QL6869","American_History_X", 1998);
    Film f2 = new Film("QL6870","La_Momia", 1998);

    @Test
    void testGetFilmTaula() {
        assertNotNull(FilmTaula.getFilmTaula());
    }

    @Test
    void testAddFilm() {
        FilmTaula.getFilmTaula().addFilm(f.getId(), f);
        FilmTaula.getFilmTaula().addFilm(f2.getId(), f2);
        assertEquals(FilmTaula.getFilmTaula().size(),2);
    }

    @Test
    void testGetFilm() {
        FilmTaula.getFilmTaula().addFilm(f.getId(), f);
        assertEquals(FilmTaula.getFilmTaula().getFilm(f.getId()
        ()),f);
    }

    @Test
    void testConteinsFilm() {
        FilmTaula.getFilmTaula().addFilm(f.getId(), f);
        assertTrue(FilmTaula.getFilmTaula().conteinsFilm(f.
        getId()));
    }

    @Test
    void testGetLista() {
        HashMap<String, Film> lista = FilmTaula.getFilmTaula()
        .getLista();
        assertNotNull(lista);
    }
}

```

Chapter 6

Konklusioak

Lan honen bidez argi gelditu da eraginkortasun-aldaketa nabaria dagoela ArrayListen ordez Hash taulak erabiltzen badira. Hura justifikatzeko, programaren bitartez jasotako datuak aurkeztuko dira.

Lehenik, ArrayListaren eta Hash taularen arteko konparaketa ikusiko da. Horretarako, bakoitzaren exekuzio-denbora neurtu da hurrengo egitean: sortu, fitxategi bakoitza irakurri eta zerrendan/taulan gehitu ez egotekotan.

Bigarrenik, Quick Sort algoritmoaren efizientzia neurtu da. Horretarako, behin taula eginda dagoela, bost proba burutu dira zerrenda ordenatu bat sortzeko eta haien exekuzio-denbora neurtu da.

6.1 Saiakerak eta emaitzak

Fitxategi kopurua	Fitxategiak	Datu lerroak	Datu berri TOT	$\bar{X}_{ArrayList}$ (s)	$\bar{X}_{HashMap}$ (s)
1	1970	8522	6306	0.3202	0.185
6	1970-1975	59946	15184	4.3986	0.4686
11	1970-1980	122812	35729	15.4578	0.7062
21	1970-1990	231331	62250	60.3115	1.1168
41	1970-2010	512169	145238	810.469	2.2012
54	1970-2023	702640	194077	1167.725	2.92

Table 6.1: Emaitzak segunduetan eta ArrayListaren eta Hash taularen konparaketa.

ArrayList probak segunduetan (s)						
Fitxategi kop.	1.proba	2.Proba	3.Proba	4.Proba	5.Proba	$\bar{X}_{ArrayList}$
1	0.304	0.359	0.338	0.279	0.321	0.3202
6	4.915	4.291	4.342	4.223	4.222	4.3986
11	15.672	15.224	15.605	15.076	15.712	15.4578
21	60.417	59.784	61.104	59.941		60.3115
41	810.469					810.469
54	1167.725					1167.725

Table 6.2: ArrayListaren proba bakoitzaren emaitza eta haien batazbestekoa

Hash taularen probak segunduetan (s)						
Fitxategi kop.	1.proba	2.Proba	3.Proba	4.Proba	5.Proba	$X_{ArrayList}$
1	0.237	0.164	0.168	0.187	0.169	0.185
6	0.445	0.478	0.474	0.468	0.478	0.4686
11	0.677	0.715	0.708	0.725	0.706	0.7062
21	1.125	1.121	1.128	1.107	1.103	1.1168
41	2.229	2.189	2.214	2.149	2.225	2.2012
54	2.948	2.960	2.910	2.887	2.895	2.92

Table 6.3: Hash taularen proba bakoitzaren emaitza eta haien batazbestekoa

6.2 ArrayList eta Hash taularen konparaketa grafikoa

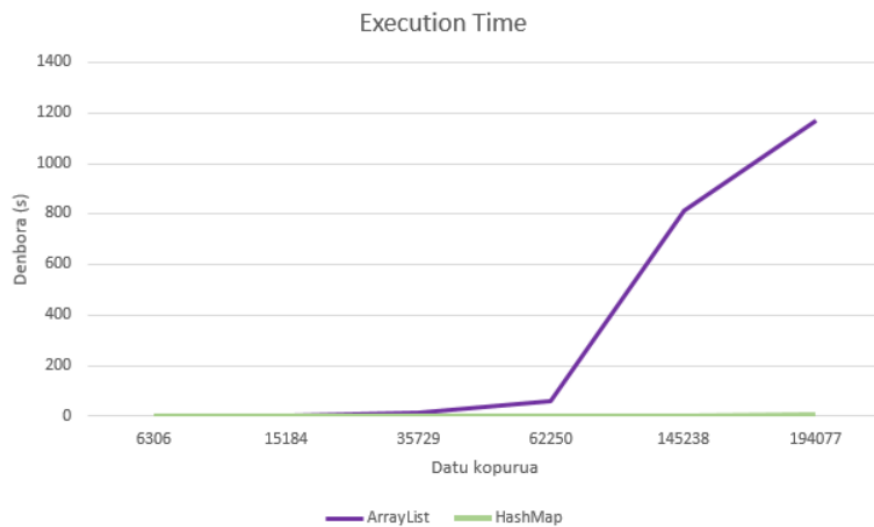


Figure 6.1: ArrayList eta Hash taularen konparaketa grafikoa

6.3 Quick Sort algoritmoaren exekuzio denbora milisegunduetan (ms)

Quick Sort algoritmoaren probak milisegunduetan (ms)						
Fitxategi kop.	1.proba	2.Proba	3.Proba	4.Proba	5.Proba	$X_{QuickSort}$
1	0.44	0.34	0.38	0.35	0.39	0.38
6	1.19	0.7	0.68	0.65	0.71	0.786
11	0.88	1.04	0.83	0.85	0.81	0.882
21	1.21	1.32	1.2	1.26	1.3	1.258
41	2.32	2.15	2.16	2.23	2.2	2.212
54	2.77	3.00	3.12	2.96	3.04	2.978

Table 6.4: Quick Sort algoritmoaren proba bakoitzaren emaitza eta haien batzbestekoa

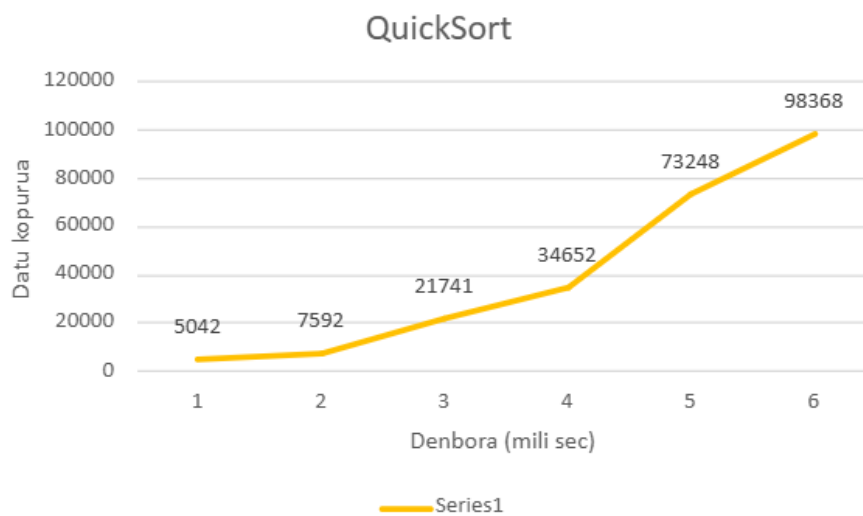


Figure 6.2: Quick Sort algoritmoaren exekuzio-denbora grafikoki